

Minime CI Workflow Architecture (Vertical Pipeline)

Unified GitHub Actions Build System — `.github/workflows/build.yml` [\[Shared\]](#)

Legend: [\[Shared\]](#) = Dual-distro / shared across targets | [\[Specific\]](#) = Single-distro, board, or libc specific

1. Trigger / Entrypoint

- Workflow File: `.github/workflows/build.yml` [\[Shared\]](#)
- Triggers: Push to `main` (filtered to `minime/**`), Pull Request, or `workflow_dispatch`

↓ *(Triggers parallel asset build jobs)*

2. Parallel assets build - bootloader and UI

2A. Bootloader Build	2B. UI Build (musl)	2C. UI Build (glibc)
build-bootloader (AMD64) • Workflow Job: <code>build-bootloader</code> [Shared] • Runner: <code>ubuntu-latest</code> • Container: <code>minime-glibc:latest</code> [Shared] • Script: <code>scripts/build-bootloader.sh</code> [Shared] • Cache Key: <code>bootloader-*</code> (hash of <code>uboot/**</code>) • Configs & Patches: - <code>minime/uboot/config/uboot.config</code> [Shared] - <code>minime/uboot/config/ddr3.defconfig</code> [Specific] (H700) - <code>minime/uboot/out/rk3566/rkbin/</code> [Specific] (RK3566) • GH Artifact: <code>bootloader-out</code>	build-ui (musl) (ARM64) • Workflow Job: <code>build-ui (musl)</code> [Shared] • Runner: <code>ubuntu-24.04-arm</code> • Container: <code>minime-musl:latest</code> [Shared] • Script: <code>scripts/build-ui.sh</code> [Shared] • Cache Key: <code>ui-musl-*</code> (hash of <code>ui/**</code>) • Submodules: - <code>minime/ui/minui/</code> [Shared] (dual <code>musl/glibc</code>) - <code>minime/ui/allium/</code> [Shared] (dual <code>musl/glibc</code>) • GH Artifact: <code>ui-musl</code> ■ <code>minui-musl-aarch64.tar.xz</code> ■ <code>allium-musl-aarch64.tar.xz</code>	build-ui (glibc) (AMD64) • Workflow Job: <code>build-ui (glibc)</code> [Shared] • Runner: <code>ubuntu-latest</code> • Container: <code>minime-glibc:latest</code> [Shared] • Script: <code>scripts/build-ui.sh</code> [Shared] • Cache Key: <code>ui-glibc-*</code> (hash of <code>ui/**</code>) • Submodules: - <code>minime/ui/minui/</code> [Shared] (dual <code>musl/glibc</code>) - <code>minime/ui/allium/</code> [Shared] (dual <code>musl/glibc</code>) • GH Artifact: <code>ui-glibc</code> ■ <code>minui-glibc-aarch64.tar.xz</code> ■ <code>allium-glibc-aarch64.tar.xz</code>

↓ *(Uploads asset artifacts to GH & Passes to OS build matrix)*

3. Artifact Transfer & Caching Layer

- Actions Used: `actions/download-artifact@v4` [\[Shared\]](#), `actions/cache/restore@v4` [\[Shared\]](#)
- Artifacts Downloaded: `bootloader-out` + matching UI artifact (`ui-musl` for Alpine, `ui-glibc` for Buildroot)
- Asset Output Caches: Bootloader Cache (`bootloader-*`), UI Cache (`ui-musl-*`, `ui-glibc-*`)
- OS Compiler Caches: CCache (`~/alpine-ccache`, `~/buildroot-ccache`) + DL Cache (`~/alpine-dl`, `~/buildroot-dl`) [\[Specific\]](#)

↓ *(Fans out to 6 parallel matrix jobs)*

4. Parallel components build - kernel, rootfs, initramfs and dtb

4A. Alpine Component Builder (musl)	4B. Buildroot Component Builder (glibc)
Alpine (musl) Component Builder (3 Jobs) • Command: <code>make -C minime/targets/alpine components BOARD=</code> [Specific] • Builder Script: <code>minime/targets/alpine/scripts/build.sh</code> [Specific] • Inputs: <code>minime/boards//</code> [Specific], <code>minime/boards/common/</code> [Shared] • Produced Component Artifacts: ■ <code>system.erofs</code> (Immutable EROFS rootfs with Mesa/Panfrost) ■ <code>Image</code> (Linux kernel binary) ■ <code>initramfs & *.dtb</code> (Device Tree Blobs)	Buildroot (glibc) Component Builder (3 Jobs) • Command: <code>make -C minime/targets/buildroot components BOARD=</code> [Specific] • Builder Script: <code>minime/targets/buildroot/scripts/build.sh</code> [Specific] • Inputs: <code>minime/boards//</code> [Specific], <code>minime/boards/common/</code> [Shared] • Produced Component Artifacts: ■ <code>system.erofs</code> (Immutable EROFS rootfs with glibc & Mali drivers) ■ <code>Image</code> (Linux kernel binary) ■ <code>initramfs & *.dtb</code> (Device Tree Blobs)

↓ *(Passes components (system.erofs, Image, DTBs) to Packaging)*

5. Image Assembly & Packaging (mkimage.sh / mkupdate.sh)

5A. Alpine Image Assembly (musl)	5B. Buildroot Image Assembly (glibc)
Alpine Image & Update Packager (musl) • Commands: - <code>make image BOARD= UI=minui allium</code> [Specific] - <code>make update BOARD= UI=minui allium</code> [Specific] • Packaging Scripts: - <code>minime/build/genassets.sh</code> [Shared] - <code>minime/build/mkimage.sh</code> [Shared] - <code>minime/build/mkupdate.sh</code> [Shared] • Produced Release Archives: ■ <code>*.img.xz</code> (Raw SD card flashable image) ■ <code>*.tar.xz</code> (OTA update package)	Buildroot Image & Update Packager (glibc) • Commands: - <code>make image BOARD= UI=minui allium</code> [Specific] - <code>make update BOARD= UI=minui allium</code> [Specific] • Packaging Scripts: - <code>minime/build/genassets.sh</code> [Shared] - <code>minime/build/mkimage.sh</code> [Shared] - <code>minime/build/mkupdate.sh</code> [Shared] • Produced Release Archives: ■ <code>*.img.xz</code> (Raw SD card flashable image) ■ <code>*.tar.xz</code> (OTA update package)

↓ *(Uploads output archives to GitHub Releases)*

6. Release & Delivery

- Workflow Step: Upload to Testing Release in `.github/workflows/build.yml` [\[Shared\]](#)
- Tool / CLI: `gh release upload testing /*.img.xz /*.tar.xz --clobber` [\[Shared\]](#)
- Destination: GitHub Releases — `testing` tag